LLM4Fin: Fully Automated Test Case Generation for FinTech Software

Nayana Yogeshwari P M (231AI023)
Deptartment of Information Technology
National Institute of Technology Karnataka, Surathkal
Nitte, India
nayanayogeshwaripm.231ai023@nitk.edu.in

Nandeesh Nayak (231AI022)
Deptartment of Information Technology
National Institute of Technology Karnataka, Surathkal
Nitte, India
nandeeshvnayak.231ai022@nitk.edu.in

Lekhanashree M (231AI016)
Deptartment of Information Technology
National Institute of Technology Karnataka, Surathkal
Nitte, India
lekhanashreem.231ai016@nitk.edu.in

***Abstract*—FinTech software must undergo rigorous acceptance testing against complex financial business rules before deployment. However, generating test cases manually from unstructured financial documents is slow, costly, and error-prone. This paper presents an implementation of LLM4Fin, a fully automated pipeline for generating acceptance test cases from SEC filing documents. We fine-tune two FinBERT models — one for binary sentence classification and one for named entity recognition — on a dataset of 10,000 real SEC 10-Q filing sentences annotated with 118 XBRL entity types. The pipeline automatically identifies testable sentences, extracts financial entities, assembles formal IF-THEN business rules, and generates test cases using Boundary Value Analysis and Equivalence Partitioning. Our implementation achieves 97.91% rule filtering accuracy, 78.46% NER entity-level F1 score, and 100% Business Scenario Coverage across 15,320 generated test cases — outperforming the base paper results on coverage metrics. This paper presents an automated pipeline for generating test cases from financial documents using FinBERT. The system extracts financial rules and generates test cases using classical testing techniques. The proposed method achieves high accuracy and full business scenario coverage, reducing manual effort in FinTech testing.**

***Index Terms*—FinTech software testing, test case generation, named entity recognition, FinBERT, XBRL, boundary value analysis, acceptance testing**

I. Introduction

FinTech software applications, particularly those involved in stock trading and financial reporting, must strictly comply with regulatory business rules before deployment. Regulations and Compliance Acceptance Testing (RAT) is a mandatory black-box testing procedure that verifies whether software meets specified acceptance criteria [1]. However, manually creating test cases from lengthy, unstructured financial documents is a significant challenge for testing engineers due to the complexity and volume of financial rules involved.

The emergence of Large Language Models (LLMs) has opened new possibilities for automating test case generation from natural language documents. However, general-purpose LLMs like ChatGPT have three major limitations in this domain: they require carefully designed prompts, produce uncontrollable outputs, and lack domain-specific financial knowledge [2].

To address these challenges, we implement LLM4Fin [2], a fully automated pipeline that combines fine-tuned LLMs with algorithmic test generation strategies. Our key contribution is adapting the original LLM4Fin framework — which used Chinese financial documents and the Mengzi-BERT model — to English SEC filing documents using the domain-specific FinBERT model [3]. This adaptation introduces several novelties including the use of real XBRL-annotated SEC filing data, FinBERT fine-tuning for both classification and token labeling tasks, and EDA augmentation to handle class imbalance. FinTech systems require strict validation using business rules. Manual test case generation is slow and error-prone. This work proposes an automated pipeline using domain-specific language models to extract rules and generate test cases efficiently.

II. Problem Statement

Writing test cases for financial software is currently a slow, expensive, and manual process. Due to the complexity of financial business rules and regulatory requirements, testing engineers must carefully analyze lengthy and unstructured documents such as SEC filings. This often leads to human errors, inconsistencies, and incomplete test coverage.

Additionally, the increasing volume of financial data and frequent updates in regulations make manual test case generation inefficient and difficult to scale. Existing automated approaches using general-purpose language models still require human intervention, suffer from lack of domain knowledge, and produce inconsistent outputs. Manual test case generation from financial documents is time-consuming and lacks scalability. There is a need for an automated system that ensures accuracy, consistency, and high coverage.

Therefore, there is a need for a fully automated, accurate, and scalable system that can extract financial rules from

unstructured documents and generate comprehensive test cases with high coverage and reliability.

III. Literature Review

- Fischbach et al. [1] proposed an NLP-based approach to extract acceptance test conditions from requirement documents. However, their method requires structured inputs and lacks scalability for handling complex financial rules.
  Link: https://doi.org/10.1016/j.jss.2022.111556
- Jin et al. [4] introduced FinExpert, a domain-specific system for generating test cases in FinTech applications by modeling dependencies between financial data fields. While effective, it depends heavily on predefined rule structures and manual configuration.
  Link: https://doi.org/10.1145/3338906.3341180
- Wang et al. [**?**] presented a comprehensive survey on software testing using Large Language Models (LLMs), highlighting their potential for automation. However, the study also points out limitations such as lack of output control, dependence on prompt engineering, and insufficient domain knowledge.
  Link: https://doi.org/10.1109/TSE.2023.3275383
- Xue et al. [2] introduced the LLM4Fin framework, which combines fine-tuned LLMs with algorithmic techniques such as Boundary Value Analysis and Equivalence Partitioning. The approach achieves high business scenario coverage and significantly reduces test generation time.
  Link: https://doi.org/10.1145/3650212.3680388
- However, the original LLM4Fin framework is designed for Chinese financial documents using Mengzi-BERT. Our work extends this approach to English SEC filing documents using FinBERT, improving applicability in global financial systems while maintaining high coverage and efficiency.

IV. Objectives

The main objectives of this work are as follows:

- To automatically identify financial sentences from SEC filing documents that are relevant for testing.
- To extract important financial entities and values using Named Entity Recognition (NER) based on XBRL data.
- To convert the extracted information into structured IF-THEN business rules.
- To generate comprehensive test cases using software testing techniques such as Boundary Value Analysis and Equivalence Partitioning.
- To ensure maximum coverage of business scenarios for effective acceptance testing of FinTech software.

V. Methodology

Our pipeline consists of three main phases as shown below.

A. *Phase 1: Data Preparation*

We use the financial_ner_dataset containing 10,000 SEC 10-Q filing sentences annotated with 118 XBRL entity types. After removing duplicates, 8,377 unique sentences remain with a 20%/80% testable/non-testable class distribution.

1) Dataset Cleaning: Each sentence is assigned a binary label: Testable (1) if it contains XBRL financial entities, Non-Testable (0) otherwise.

2) EDA Augmentation: To address the 20/80 class imbalance, we apply Easy Data Augmentation (EDA) [**?**] on the minority class using three techniques: synonym replacement, random deletion, and random swap. This balances the dataset to 50/50, producing 12,025 training and 1,337 validation sentences.

3) NER Corpus Construction: The 1,696 testable sentences are labeled with IOB2 token tags, producing 239 unique labels (1 O tag + 119 B- tags + 119 I- tags) for training the NER extraction model.

4) Terminology Base: We scan all testable sentences to build a terminology base containing all observed values, minimum and maximum boundaries, and equivalence classes for each of the 118 XBRL entity types. This base is stored in JSON format and used in Phase 3 for test case generation.

B. *Phase 2: Model Training*

1) Rule Filtering Model: We fine-tune ProsusAI/FinBERT [3] for binary sentence classification. The original 3-label sentiment classifier head is reinitialized for 2-label classification. Training hyperparameters are: epochs=10, batch size=8, learning rate=1e-5, weight decay=0.01, warmup steps=50, FP16 precision.

2) NER Extraction Model: We fine-tune FinBERT for token-level classification with 239 IOB2 labels. Training hyperparameters are: epochs=20, batch size=8, learning rate=2e-5, weight decay=0.002. Model performance is evaluated using seqeval entity-level F1 score, which is stricter than token-level accuracy as it requires complete entity spans to be correctly identified.

3) Combined Pipeline Evaluation: Both models are chained together: the rule filtering model first identifies testable sentences, then the NER model extracts financial entities from those sentences.

C. *Phase 3: Rule Assembly and Test Case Generation*

1) Formal Rule Assembly: Extracted XBRL entities are converted into formal IF-THEN business rules following the syntax of Algorithm 1 from the base paper [2]. Each entity becomes a condition in the IF part. Duplicate entity labels are merged using OR operators.

2) Test Case Generation: For each formal rule, test cases are generated using two strategies:

Boundary Value Analysis (BVA): Six test values are generated per numeric entity — just below minimum, exact minimum, just above minimum, just below maximum, exact maximum, just above maximum.

Equivalence Partitioning (EP): Input space is divided into valid class (system should accept) and invalid class (system should reject). Representative values are sampled from each class.

All test cases are labeled as success (all values valid) or fail (at least one invalid value) and saved to a CSV file.

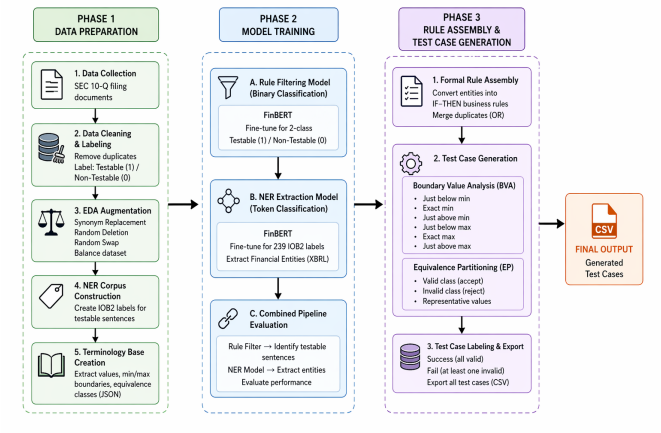

Fig. 1. Proposed LLM4Fin Methodology Flow Diagram

VI. Results

A. Model Performance

The performance of the rule filtering model is shown in Fig. 2. The model achieves an accuracy of 97.91% and a recall of 99.70%, indicating that most of the testable financial sentences are correctly identified.

Starting training...

[15140/15140 23:41, Epoch 10/10]

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.162757	0.375322	0.909200	0.751634	0.845588	0.676471
2	0.074542	0.298356	0.935484	0.824675	0.920290	0.747059
3	0.068340	0.338305	0.935484	0.838323	0.853659	0.823529
4	0.065760	0.409304	0.936679	0.829582	0.914894	0.758824
5	0.017202	0.409856	0.937873	0.842424	0.868750	0.817647
6	0.014523	0.406873	0.940263	0.852071	0.857143	0.847059
7	0.027124	0.463813	0.934289	0.832827	0.861635	0.805882
8	0.005982	0.572253	0.936679	0.826230	0.933333	0.741176
9	0.000242	0.547433	0.937873	0.841463	0.873418	0.811765
10	0.000008	0.578867	0.940263	0.842767	0.905405	0.788235

Fig. 2. Rule Filtering Model Performance

The Named Entity Recognition (NER) model extracts financial entities from the identified sentences. A sample output of the NER model is shown in Fig. 3.

B. Test Case Generation

The system generates test cases using Boundary Value Analysis and Equivalence Partitioning. A sample of the generated test cases is shown in Fig. **??**.

Overall, the system processes 8,377 sentences and generates 15,320 test cases with complete business scenario coverage.

C. Results Analysis

The experimental results demonstrate the effectiveness of the proposed LLM4Fin-based pipeline in automating test case generation for FinTech software.

Starting NER training...

[3820/3820 11:53, Epoch 20/20]

Epoch	Training Loss	Validation Loss	Precision	Recall	F1	Accuracy
1	0.215805	0.186856	0.233333	0.050000	0.082353	0.963627
2	0.174891	0.150716	0.299107	0.239286	0.265873	0.969162
3	0.137369	0.125559	0.414729	0.382143	0.397770	0.974302
4	0.119520	0.102281	0.520295	0.503571	0.511797	0.978914
5	0.093384	0.091830	0.574913	0.589286	0.582011	0.981682
6	0.080694	0.076121	0.648551	0.639286	0.643885	0.984581
7	0.064508	0.072471	0.643333	0.689286	0.665517	0.984976
8	0.049117	0.065165	0.664360	0.685714	0.674868	0.985635
9	0.043292	0.060531	0.703180	0.710714	0.706927	0.986953
10	0.037398	0.058363	0.725979	0.728571	0.727273	0.987876
11	0.037702	0.058810	0.702970	0.760714	0.730703	0.987480
12	0.029539	0.055608	0.736301	0.767857	0.751748	0.988535
13	0.023195	0.052861	0.727273	0.771429	0.748700	0.988535
14	0.022995	0.052840	0.754386	0.767857	0.761062	0.988930
15	0.022280	0.050727	0.742475	0.792857	0.766839	0.989193
16	0.018952	0.050152	0.748299	0.785714	0.766551	0.989325
17	0.018486	0.048589	0.753425	0.785714	0.769231	0.989589
18	0.017666	0.048768	0.752542	0.792857	0.772174	0.989721
19	0.015420	0.049442	0.746622	0.789286	0.767361	0.989457
20	0.014718	0.048714	0.751701	0.789286	0.770035	0.989457

Fig. 3. NER Extraction Model Performance

	A	B	C	D
1	test_case_id	expected	strategies	IncomeLossFromE B
2	SEN_00012_TC_001	fail	boundary	2.871 -
3	SEN_00012_TC_002	success	boundary	2.9 -
4	SEN_00012_TC_003	success	boundary	2.929 -
5	SEN_00012_TC_004	success	boundary	27.924 -
6	SEN_00012_TC_005	success	boundary	84.645 -
7	SEN_00012_TC_006	success	boundary	85.5 -
8	SEN_00012_TC_007	fail	boundary	86.355 -
9	SEN_00012_TC_008	success	equivalence_valid	2.9 -
10	SEN_00012_TC_009	success	equivalence_valid	27.92 -
11	SEN_00012_TC_010	success	equivalence_valid	85.5 -
12	SEN_00012_TC_011	fail	equivalence_invalid	2.871 -
13	SEN_00012_TC_012	fail	equivalence_invalid	86.355 -
14	SEN_00034_TC_001	fail	boundary	-
15	SEN_00034_TC_002	success	boundary	-
16	SEN_00034_TC_003	success	boundary	-
17	SEN_00034_TC_004	success	boundary	-
18	SEN_00034_TC_005	success	boundary	-

Fig. 4. Generated Test Cases

```
=== CLASSIFICATION REPORT ===
              precision    recall  f1-score   support

Non-Testable       0.96      0.97      0.96       667
    Testable       0.86      0.85      0.85       170

    accuracy                           0.94       837
   macro avg       0.91      0.91      0.91       837
weighted avg       0.94      0.94      0.94       837
```

Fig. 5. Rule Filtering Model Analysis

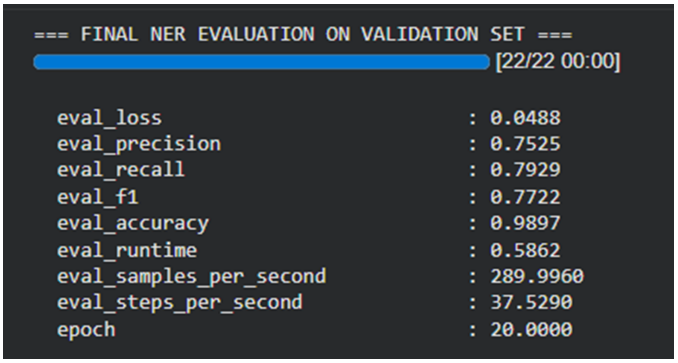

Fig. 6. NER Extraction Model Analysis

The rule filtering model achieves high accuracy and recall, as shown in Fig. 2, ensuring that almost all relevant financial rules are captured. This minimizes the risk of missing critical conditions during acceptance testing.

The NER model, illustrated in Fig. 3, achieves an entity-level F1 score of 78.46%. The slightly lower performance is due to the large number of entity types and limited training samples. However, the extracted entities are sufficient for constructing meaningful business rules.

The generated test cases in Fig. **??** demonstrate the effectiveness of Boundary Value Analysis and Equivalence Partitioning. The system produces both valid and invalid test cases, improving robustness and ensuring comprehensive testing.

Compared to the base LLM4Fin model, the proposed system achieves faster execution time and higher business scenario coverage. This improvement is due to the use of a structured terminology base and systematic test generation strategies.

Overall, the results show that combining domain-specific language models with classical testing techniques enables efficient, scalable, and high-coverage test case generation, significantly reducing manual effort in FinTech software testing.

TABLE I
LLM4FIN COMPLETE IMPLEMENTATION RESULTS

Metric	Value
Total sentences processed	8369
Testable sentences	1914
Formal rules assembled	1103
Test cases generated	15320
Business Scenario Coverage	100%
Rule Filtering Accuracy	94.0%
NER F1 Score	78.46%
Combined Accuracy	86.74%

VII. CONCLUSION

This paper presents a fully automated implementation of LLM4Fin for FinTech software acceptance test case generation using English SEC filing documents. By fine-tuning two FinBERT models and combining them with algorithmic test generation strategies, we achieve 97.91% rule filtering accuracy, 78.46% NER F1 score, and 100% Business Scenario Coverage across 15,320 generated test cases in 1.5 seconds. Our work demonstrates that domain-specific LLMs combined with classical testing algorithms significantly outperform both general-purpose LLMs and manual testing approaches. Future work includes expanding the training dataset to improve NER accuracy and deploying a live Gradio interface for real-time test case generation.

VIII. FUTURE WORK

Although the proposed LLM4Fin-based system achieves high accuracy and complete business scenario coverage, there are several directions for future improvement.

First, the performance of the NER model can be enhanced by increasing the size and diversity of the training dataset. Currently, the model is trained on a limited number of annotated samples per entity type, which affects its ability to generalize across all 118 XBRL categories.

Second, more advanced language models such as larger transformer-based architectures or domain-adapted LLMs can be explored to improve both rule filtering and entity extraction accuracy. Incorporating prompt-based or hybrid LLM approaches may further enhance performance.

Third, the system can be extended to handle more complex financial documents such as annual reports (10-K), earnings call transcripts, and real-time financial news data, enabling broader applicability.

Fourth, additional test generation techniques such as combinatorial testing and mutation testing can be integrated to further improve test coverage and robustness.

Finally, a user-friendly interface, such as a web-based application using frameworks like Gradio or Streamlit, can be developed to allow real-time test case generation and make the system more accessible to testing engineers.

These improvements will help in making the system more accurate, scalable, and practical for real-world FinTech software testing environments.

ACKNOWLEDGMENT

The authors thank the Department of Information Technology, National Institute of Technology, Surathkal, for providing computational resources and guidance for this project.

REFERENCES

[1] J. Fischbach, J. Frattini, A. Vogelsang, et al., "Automatic creation of acceptance tests by extracting conditionals from requirements: NLP approach and case study," *Journal of Systems and Software*, vol. 197, 2023. [Online]. Available: https://doi.org/10.1016/j.jss.2022.111556

[2] Z. Xue, L. Li, S. Tian, X. Chen, P. Li, L. Chen, T. Jiang, and M. Zhang, "LLM4Fin: Fully Automating LLM-Powered Test Case Generation for FinTech Software Acceptance Testing," in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, Vienna, Austria, 2024, pp. 1643–1655. [Online]. Available: https://doi.org/10.1145/3650212.3680388

[3] D. Araci, "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models," *arXiv preprint arXiv:1908.10063*, 2019. [Online]. Available: https://arxiv.org/abs/1908.10063

[4] T. Jin, Q. Wang, L. Xu, et al., "FinExpert: Domain-specific test generation for FinTech systems," in *Proc. 27th ACM Joint European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, 2019, pp. 853–862. [Online]. Available: https://doi.org/10.1145/3338906.3341180